MERN Stack Course – Session 4

REST API Security: JWT Authentication, Bcrypt Hashing & Route Protection

Dr. Abdelweheb GUEDDES – École Polytechnique de Sousse – 2026–2027

Objective	Secure passwords using `bcryptjs`, issue and verify JSON Web Tokens (JWT), and protect endpoints using Express middlewares for authentication and Role-Based Access Control (RBAC).	**Duration**	3 hours (hands-on laboratory)
Prerequisites	Session 3 completed (MongoDB Atlas persistence, Mongoose schemas/models, async operations).	**Deliverable**	Individual PDF laboratory report submitted on the university portal before Session 5.

How to navigate this document. Follow the active learning loop:

1. **Predict**: anticipate security vulnerabilities, cryptographic signature verification, and middleware flow **before** running code.
2. **Your Turn**: implement password salting, token issuance, and protected routes in Node.js/Express, then test with Postman.
3. **Checkpoint**: inspect expected HTTP response codes (`200`, `201`, `401`, `403`, `409`) and check your MongoDB Atlas collection to ensure zero plaintext passwords exist.

1 Why Security Matters: Moving from Open to Protected APIs

Following Session 3, our REST API persists data in MongoDB Atlas and models relationships between articles and authors. However, in its current state, two **critical security vulnerabilities** remain:

1. **Total Identity Impersonation:** When creating an article via `POST /api/articles`, the client sends `authorId` in the request body. Anyone can impersonate any author in the database!
2. **Lack of Access Control:** The `PUT /api/articles/:id` and `DELETE /api/articles/:id` endpoints execute operations without checking if the requester owns the article or holds administrator rights.
3. **Unprotected Passwords:** If the database is compromised, plaintext passwords grant direct unauthorized access to all user accounts.

Predict. Why is it forbidden to use reversible encryption (like AES) or fast unsalted hash algorithms (like raw MD5 or SHA-256) for storing passwords? What happens if an attacker uses precomputed rainbow tables?

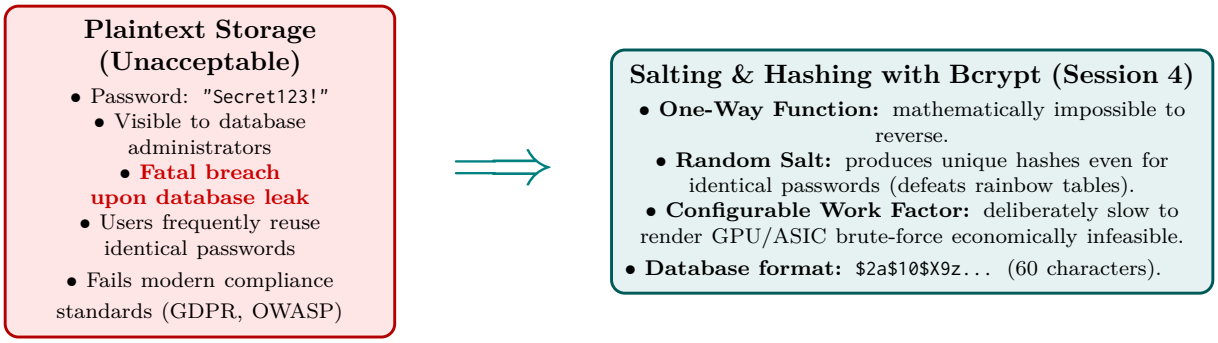


Figure 1: Comparison between vulnerable plaintext storage and salted cryptographic hashing with Bcrypt.

Concept: Salting and Hashing. A hash function computes a deterministic, fixed-length fingerprint from an input string. Unlike encryption, hashing is **irreversible**. To prevent dictionary attacks, `bcryptjs` appends a cryptographically secure random string called a **Salt** before executing computational rounds. During login, `bcrypt.compare()` extracts the salt from the stored hash, rehashes the submitted candidate with that same salt, and performs constant-time comparison to prevent timing side-channel attacks.

2 How JSON Web Tokens (JWT) Work

In modern decoupled architectures (React Single Page Application + Node/Express REST API), the server must remain **Stateless**. Sessions stored in server RAM do not scale horizontally.

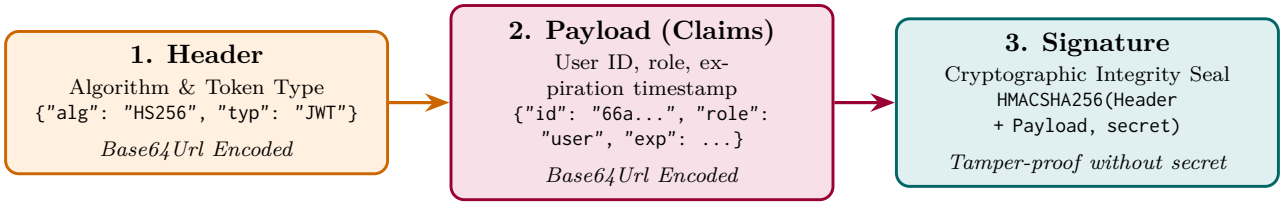


Figure 2: Three-part structure of a JSON Web Token: `Header.Payload.Signature`.

Crucial Rule Regarding Payload Confidentiality: A JWT payload is merely **encoded** in Base64Url, **not encrypted**! Anyone can inspect it on https://jwt.io. You must **NEVER** store passwords, credit card numbers, or confidential data inside a JWT payload.

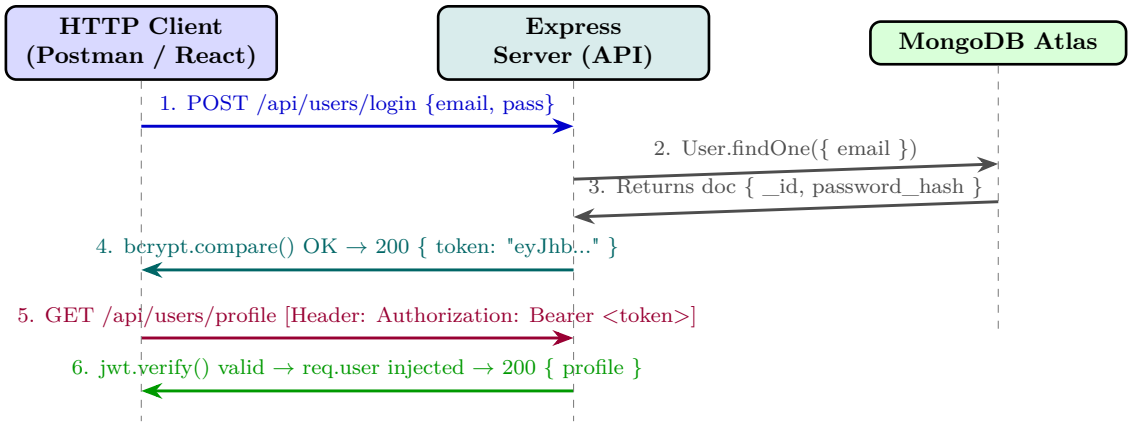


Figure 3: Complete authentication and stateless request validation lifecycle with JWT.

3 Express Middlewares as Security Gatekeepers

An Express **middleware** is a function with signature `(req, res, next)` that intercepts incoming HTTP requests before they reach controller handlers.

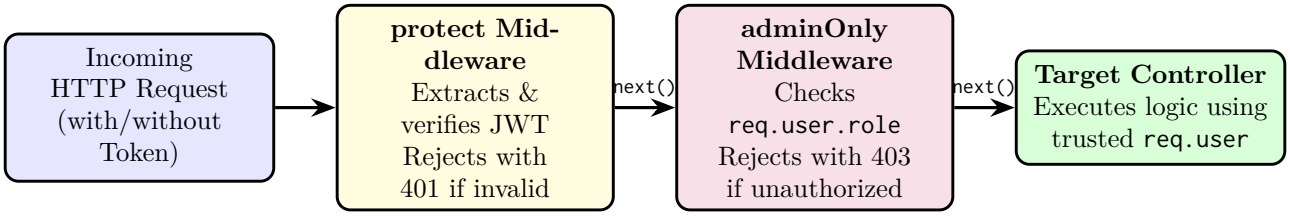


Figure 4: Sequential authorization middleware pipeline in Express.

Concept: The Standard `Authorization: Bearer <token>` Header. Industry standards transmit access tokens via the HTTP `Authorization` request header using the `Bearer` scheme. In Express, extract the token with:

```
const token = req.headers.authorization.split(' ')[1];
```

When verification succeeds, the middleware injects user details directly into the request object:

`req.user = await User.findById(decoded.id).select('-password');`. Subsequent controllers access the verified caller identity with certainty!

4 Hands-on Laboratory: Building the Authentication System

In this guided lab, we upgrade our application architecture with hashed passwords, cryptographic tokens, and protected write endpoints.

4.1 Step 1: Installing Security Dependencies

Open a terminal in your project directory and install the required modules:

```
npm install bcryptjs jsonwebtoken
```

- `bcryptjs`: Pure JavaScript implementation of the Bcrypt password hashing algorithm (zero native C++ compilation dependencies).
- `jsonwebtoken`: Reference library for signing (`sign`) and verifying (`verify`) JWTs.

4.2 Step 2: Configuring the Application Secret (`.env`)

Add the `JWT_SECRET` environment variable to your `.env` file:

```
PORT=3000
MONGODB_URI=mongodb+srv://admin_mern:<password>@cluster0.xxxxx.mongodb.net/mern_auth_db?
    retryWrites=true&w=majority
JWT_SECRET=polytechnique_sousse_super_secret_jwt_key_2026_securite
```

4.3 Step 3: Updating the User Schema (`userModel.js`)

We enforce password constraints (`minlength: 6`), a role enum (`'user'` or `'admin'`), and email validation.

➥ File `src/models/userModel.js` – run with `node src/server.js`

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Name is required'],
      trim: true
    },
    email: {
      type: String,
      required: [true, 'Email is required'],
      unique: true,
      lowercase: true,
      trim: true,
      match: [/^[^\s@]+@[^\s@]+\.[^\s@]+$/, 'Invalid email format']
    },
    password: {
      type: String,
      required: [true, 'Password is required'],
      minlength: [6, 'Password must be at least 6 characters long']
    },
    role: {
      type: String,
      enum: {
        values: ['user', 'admin'],
        message: '{VALUE} is not a valid role'
      },
      default: 'user'
    },
    age: {
      type: Number,
      min: [4, 'Minimum allowed age is 4']
    }
  },
```

```
  { timestamps: true }
);

module.exports = mongoose.model('User', userSchema);
```

4.4 Step 4: Security Middlewares (`authMiddleware.js`)

Create the `src/middlewares/` directory and the `authMiddleware.js` file:

➥ File `src/middlewares/authMiddleware.js` – run with `node src/server.js`

```
const jwt = require('jsonwebtoken');
const User = require('../models/userModel');

// 1. Route protection: verifies token validity
const protect = async (req, res, next) => {
  let token;

  if (
    req.headers.authorization &&
    req.headers.authorization.startsWith('Bearer ')
  ) {
    try {
      token = req.headers.authorization.split(' ')[1];
      const decoded = jwt.verify(token, process.env.JWT_SECRET);

      req.user = await User.findById(decoded.id).select('-password');
      if (!req.user) {
        return res.status(401).json({ message: 'Not authorized, user not found' });
      }

      return next();
    } catch (err) {
      return res.status(401).json({ message: 'Not authorized, invalid or expired token' })
    ;
    }
  }

  return res.status(401).json({ message: 'Not authorized, no token provided' });
};

// 2. Role-Based Access Control (RBAC): admin only
const adminOnly = (req, res, next) => {
  if (req.user && req.user.role === 'admin') {
    return next();
  }
  return res.status(403).json({ message: 'Access denied: administrator privileges required
    ' });
};

module.exports = { protect, adminOnly };
```

4.5 Step 5: Authentication Controller (`userController.js`)

We implement:

1. A helper function `generateToken(id, role)` issuing 24-hour JWTs.
2. User registration (`registerUser`): salting and hashing with `bcrypt.genSalt(10)`.
3. Login (`loginUser`): constant-time password verification with `bcrypt.compare()`.
4. Profile lookup (`getUserProfile`): returns verified `req.user`.

➥ File `src/controllers/userController.js` – run with `node src/server.js`

```
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
```

```
const User = require('../models/userModel');

const generateToken = (id, role) => {
  return jwt.sign({ id, role }, process.env.JWT_SECRET, {
    expiresIn: '1d'
  });
};

// POST /api/users/register (Registration)
const registerUser = async (req, res) => {
  try {
    const { name, email, password, age, role } = req.body;

    if (!name || !email || !password) {
      return res.status(400).json({ message: 'Name, email, and password are required' });
    }

    const userExists = await User.findOne({ email });
    if (userExists) {
      return res.status(409).json({ message: 'Email is already registered' });
    }

    const salt = await bcrypt.genSalt(10);
    const hashedPassword = await bcrypt.hash(password, salt);

    const user = await User.create({
      name,
      email,
      password: hashedPassword,
      age,
      role
    });

    res.status(201).json({
      _id: user._id,
      name: user.name,
      email: user.email,
      role: user.role,
      token: generateToken(user._id, user.role)
    });
  } catch (err) {
    if (err.name === 'ValidationError') {
      return res.status(400).json({ message: err.message });
    }
    res.status(500).json({ message: err.message });
  }
};

// POST /api/users/login (Authentication)
const loginUser = async (req, res) => {
  try {
    const { email, password } = req.body;

    if (!email || !password) {
      return res.status(400).json({ message: 'Email and password are required' });
    }

    const user = await User.findOne({ email });
    if (!user) {
      return res.status(401).json({ message: 'Invalid credentials' });
    }

    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) {
      return res.status(401).json({ message: 'Invalid credentials' });
    }
```

```
    res.status(200).json({
      _id: user._id,
      name: user.name,
      email: user.email,
      role: user.role,
      token: generateToken(user._id, user.role)
    });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

// GET /api/users/profile (Protected profile)
const getUserProfile = async (req, res) => {
  try {
    res.status(200).json(req.user);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

// GET /api/users (Admin only)
const getAllUsers = async (req, res) => {
  try {
    const users = await User.find().select('-password');
    res.status(200).json(users);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

module.exports = { registerUser, loginUser, getUserProfile, getAllUsers };
```

4.6 Step 6: Secure Article Controller (`articleController.js`)

Article ownership is now extracted directly from verified credentials via `req.user._id`. Updates and deletions check ownership or administrator privileges.

➥ File `src/controllers/articleController.js` – run with `node src/server.js`

```
const Article = require('../models/articleModel');

// POST /api/articles (Protected creation)
const createArticle = async (req, res) => {
  try {
    const { title, content } = req.body;

    if (!title || !content) {
      return res.status(400).json({ message: 'Title and content are required' });
    }

    const newArticle = await Article.create({
      title,
      content,
      author: req.user._id
    });

    const populated = await newArticle.populate('author', 'name email role');
    res.status(201).json(populated);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

// PUT /api/articles/:id (Protected update: author or admin)
```

```
const updateArticle = async (req, res) => {
  try {
    const article = await Article.findById(req.params.id);
    if (!article) {
      return res.status(404).json({ message: 'Article not found' });
    }

    if (article.author.toString() !== req.user._id.toString() && req.user.role !== 'admin'
    ) {
      return res.status(403).json({
        message: 'Forbidden: you are not the author of this article'
      });
    }

    article.title = req.body.title || article.title;
    article.content = req.body.content || article.content;
    const updated = await article.save();
    const populated = await updated.populate('author', 'name email role');

    res.status(200).json(populated);
  } catch (err) {
    if (err.name === 'CastError') {
      return res.status(400).json({ message: 'Invalid article ID format' });
    }
    res.status(500).json({ message: err.message });
  }
};

// DELETE /api/articles/:id (Protected delete: author or admin)
const deleteArticle = async (req, res) => {
  try {
    const article = await Article.findById(req.params.id);
    if (!article) {
      return res.status(404).json({ message: 'Article not found' });
    }

    if (article.author.toString() !== req.user._id.toString() && req.user.role !== 'admin'
    ) {
      return res.status(403).json({
        message: 'Forbidden: you are not the author of this article'
      });
    }

    await Article.findByIdAndDelete(req.params.id);
    res.status(200).json({ message: 'Article deleted successfully', id: req.params.id });
  } catch (err) {
    if (err.name === 'CastError') {
      return res.status(400).json({ message: 'Invalid article ID format' });
    }
    res.status(500).json({ message: err.message });
  }
};
```

4.7 Step 7: Route Wiring (`userRoutes.js` and `articleRoutes.js`)

Mount the security middlewares in the modular routers:

➥ File `src/routes/userRoutes.js` – run with `node src/server.js`

```
const express = require('express');
const router = express.Router();
const {
  registerUser, loginUser,
  getUserProfile, getAllUsers
} = require('../controllers/userController');
const { protect, adminOnly } = require('../middlewares/authMiddleware');
```

```
// Public authentication routes
router.post('/register', registerUser);
router.post('/login', loginUser);

// Private profile route protected by JWT
router.get('/profile', protect, getUserProfile);

// Administrator-only route (Valid JWT + admin role)
router.get('/', protect, adminOnly, getAllUsers);

module.exports = router;
```

➥ File `src/routes/articleRoutes.js` – run with `node src/server.js`

```
const express = require('express');
const router = express.Router();
const {
  getAllArticles, getArticleById,
  createArticle, updateArticle,
  deleteArticle
} = require('../controllers/articleController');
const { protect } = require('../middlewares/authMiddleware');

// Public read routes
router.get('/', getAllArticles);
router.get('/:id', getArticleById);

// Protected write operations
router.post('/', protect, createArticle);
router.put('/:id', protect, updateArticle);
router.delete('/:id', protect, deleteArticle);

module.exports = router;
```

4.8 Step 8: Methodical Postman Verification Suite

Execute this test sequence in Postman to validate the security flow:

Method	Endpoint	Headers / Request Specifics	Expected Status
`POST`	`/api/users/register`	JSON body with name, email, password ($\geq$ 6 chars)	**201 Created**
`POST`	`/api/users/login`	Valid registered credentials → Copy returned `token`	**200 OK**
`GET`	`/api/users/profile`	**Without** `Authorization` header	**401 Unauthorized**
`GET`	`/api/users/profile`	Header: `Authorization: Bearer <your_token>`	**200 OK**
`POST`	`/api/articles`	Token header + Body { title, content }	**201 Created**
`GET`	`/api/users`	Standard token with `role: "user"` → RBAC check	**403 Forbidden**

Checkpoint. Open the MongoDB Atlas dashboard (*Browse Collections*) and view your user document. Confirm that `password` starts with `$2a$10$` and contains exactly 60 characters. If plaintext is visible, your hashing logic is flawed!

5 Independent Exercises (Graded Laboratory Work)

Complete the following exercises individually and document your results in your report.

Exercise 1: Secure Password Update (6 points) ★★

Implement a protected endpoint enabling authenticated users to update their password:

1. Declare the route `PUT /api/users/update-password` protected by the `protect` middleware.
2. Expect `oldPassword` and `newPassword` in the request body.
3. Verify that `oldPassword` matches the stored hash via `bcrypt.compare()`. Return `401 Unauthorized` with an explicit message if it fails.
4. If verified, generate a new salt, hash `newPassword`, save the document, and return `200 OK`.
5. Verify in Postman by logging in with the new password.

Exercise 2: Personal Articles Filter & Privilege Escalation Defense (8 points) ★★

1. Create a protected route `GET /api/articles/my-articles` declared **before** `GET /api/articles/:id` in `articleRoutes.js`.
2. In the handler `getMyArticles`, return only articles authored by the caller:

```
const articles = await Article.find({ author: req.user._id });
```

3. **Privilege Escalation Defense:** Prevent users from creating admin accounts via public registration (force `role = 'user'` inside `registerUser`).
4. Add an admin-only endpoint `PUT /api/users/:id/role` allowing administrators to promote users to the `'admin'` role.

Exercise 3: Explicit Token Expiration Handling (TokenExpiredError) (6 points) ★★★

When a JWT expires according to `expiresIn`, `jwt.verify()` throws a `TokenExpiredError`.

1. In `protect` (`authMiddleware.js`), inspect caught errors in the `catch (err)` block.
2. If `err.name === 'TokenExpiredError'`, return status `401 Unauthorized` with:

```
{ "message": "Session expired, please log in again" }
```

3. Temporarily change `expiresIn` to `'10s'` in `generateToken`. Log in, wait 15 seconds, and send a request to `/api/users/profile` to verify expiration handling. Restore `'1d'` afterward.

Laboratory Report Submission Guidelines

Each student submits **a single individual PDF report** on the university platform including:

1. Your reasoned response to the **Predict** callout in Section 1.
2. Atlas screenshot showing the hashed password format (`$2a$10$...`).
3. Postman screenshots demonstrating:
   - Registration and login with JWT acquisition;
   - Rejection of unauthorized requests without token (`401 Unauthorized`);
   - Successful request with `Authorization: Bearer <token>`;
   - RBAC rejection of admin routes for standard accounts (`403 Forbidden`);
   - Verified tests for password update (Exercise 1) and personal articles (Exercise 2).
4. Full source code of your middlewares and enriched controllers.

Submission Deadline: Eve of Session 5 at 23:59 on the university portal. Late submissions will incur penalties.

6 Summary

Six Core Takeaways

1. Passwords must **never** be stored in plaintext: `bcryptjs` applies irreversible one-way hashing with random salts to defeat dictionary attacks.
2. **JSON Web Tokens (JWT)** provide stateless authentication suited for decoupled frontend/backend architectures.
3. A JWT consists of three dot-separated components: **Header.Payload.Signature**. Only the server holding `JWT_SECRET` can verify its signature.
4. Sensitive credentials must never appear in the payload because it is merely Base64Url encoded and readable by anyone.
5. **Express middlewares** act as perimeter guards validating token integrity (`protect`) and user privileges (`adminOnly`).
6. The verified user identity is attached to `req.user`, eliminating spoofing risks in creation and modification endpoints.

I want to...	**I write...**
Generate a 10-round random salt	`const salt = await bcrypt.genSalt(10);`
Hash a plaintext password with salt	`const hash = await bcrypt.hash(password, salt);`
Verify a candidate password during login	`const isMatch = await bcrypt.compare(pass, user.password);`
Sign a 24-hour access JWT	`jwt.sign({ id, role }, process.env.JWT_SECRET, { expiresIn: '1d' });`
Verify and decode a JWT	`const decoded = jwt.verify(token, process.env.JWT_SECRET);`
Extract the token from request headers	`const token = req.headers.authorization.split(' ')[1];`
Exclude the password from database queries	`await User.findById(id).select('-password');`
Protect an endpoint with middleware	`router.post('/articles', protect, createArticle);`

Self-Assessment Checklist

☐	Explain why plaintext password storage is a critical vulnerability	Section 1
☐	Understand the role of random salts and bcrypt work factors	Section 1
☐	Describe the 3 parts of a JWT and cryptographic signature verification	Section 2
☐	Configure `JWT_SECRET` securely in environment variables	Section 4
☐	Implement an Express middleware validating the `Authorization: Bearer` header	Section 4
☐	Inject authenticated caller identity into `req.user` to prevent impersonation	Section 4
☐	Implement Role-Based Access Control (RBAC) with `adminOnly`	Section 4
☐	Validate security HTTP status codes (`401`, `403`) in Postman	Section 5

7 Session Concept Reference

1. Bcrypt Cryptographic Methods (`bcryptjs`)

Bcrypt Method	Technical Role	Code Pattern
`bcrypt.genSalt(rounds)`	Generates a cryptographically secure salt ($2^{10} = 1024$ rounds)	`const salt = await bcrypt.genSalt(10);`
`bcrypt.hash(pass, salt)`	Produces the irreversible 60-character hash fingerprint	`const hash = await bcrypt.hash(pwd, salt);`
`bcrypt.compare(txt, hash)`	Rehashes candidate password with stored salt in constant time	`const ok = await bcrypt.compare(input, user.pass);`

2. JSON Web Token Methods & Options (`jsonwebtoken`)

JWT Method / Option	Technical Meaning	Concrete Usage
`jwt.sign(payload, secret, opt)`	Combines Header and Payload, computes HMAC-SHA256 signature	`jwt.sign({ id: user._id }, secret, { expiresIn: '1d' });`
`jwt.verify(token, secret)`	Verifies signature authenticity and returns decoded claims	`const decoded = jwt.verify(token, secret);`
`expiresIn: '1d' / '2h' / '30m'`	Defines the token lifespan from issuance	`{ expiresIn: '2h' }` (expires in 2 hours)
`decoded.id` / `decoded.role`	Claims extracted from the verified payload	`req.user = await User.findById(decoded.id);`

3. Security-Related HTTP Status Codes

HTTP Status	Official Meaning	API Practical Scenario
200 OK	Request succeeded	Successful login, profile retrieved
201 Created	Resource created successfully	User registered, article created
400 Bad Request	Invalid payload format	Password too short, missing fields, regex failure
401 Unauthorized	Missing or invalid authentication	Missing token, invalid signature, wrong password
403 Forbidden	Authenticated but insufficient permissions	Standard user requesting admin endpoint
404 Not Found	Resource does not exist	User or article not found
409 Conflict	State conflict	Duplicate email registration attempt
500 Internal Server	Unexpected server exception	Unhandled database crash, unhandled runtime error

4. REST API Security Best Practices

Security Principle	Rationale	Course Implementation
Never return password hashes	Prevents hash leakage in API responses	`.select('-password')` in queries
Strong signing secret	Prevents brute-forcing token signatures	Long key stored in `.env`
Never trust client ID input	Author identity must be extracted from the token	`author: req.user._id` instead of `req.body.authorId`
Short token lifespan	Minimizes exploitation window if compromised	`expiresIn: '1d'` on signature

Coming Up Next (Session 5: Introduction to Modern Front-End with React & Vite). Now that our REST API is fully functional and secured with JWT authentication, we are ready to build the user interface! We will explore creating a React application with Vite, component-driven design, JSX syntax, managing state with the `useState` hook, and communicating with our secure backend API.